

5주차 - 예외처리

날짜 2026년 5월 4일

1. 예외(Exception)란?

프로그램 실행 중 발생하는 오류를 **예외(Exception)** 라고 한다.

예외가 발생하면 프로그램이 중간에 멈추거나 강제 종료될 수 있다.

예외가 왜 발생할까?

컴퓨터는 우리가 시킨 명령을 정확하게 수행한다. 그런데 프로그램 실행 도중 예상하지 못한 상황이 생기면 오류가 발생한다.

예시:

- 숫자를 입력해야 하는데 문자를 입력함
- 리스트 범위를 벗어난 번호를 사용함
- 존재하지 않는 파일을 열려고 함
- 0으로 나누기 함

2. 예외가 발생하는 예시

예시 1 - 숫자가 아닌 값 입력

```
num = int(input("숫자 입력: "))  
print(num)
```

실행:

숫자 입력: abc

결과:

ValueError

오류 발생 원인

```
int()
```

는 문자열을 숫자로 바꾸는 함수이다.

그런데 "abc" 는 숫자로 바꿀 수 없기 때문에 오류 발생.

예시 2 - 0으로 나누기

```
print(10 / 0)
```

결과:

```
ZeroDivisionError
```

컴퓨터에서는 어떤 수도 0으로 나눌 수 없다.

예시 3 - 리스트 범위 초과

```
numbers = [10, 20, 30]
```

```
print(numbers[5])
```

결과:

```
IndexError
```

리스트에는 0,1,2 인덱스만 존재하는데 5번을 요청했기 때문.

3. 예외 처리를 왜 사용할까?

예외 처리를 하지 않으면 프로그램이 바로 종료된다.

예외 처리를 하면:

- 프로그램 종료 방지 가능

- 사용자에게 친절한 안내 가능
- 안정적인 프로그램 제작 가능

예외 처리 없는 경우

```
num = int(input("숫자 입력: "))  
print(10 / num)  
  
print("프로그램 종료")
```

입력:

0

결과:

ZeroDivisionError

아래 코드 실행 안 됨:

```
print("프로그램 종료")
```

4. try-except 문

예외를 처리하는 가장 기본적인 문법

기본 구조

```
try:  
    실행할 코드  
  
except:  
    예외 발생 시 실행할 코드
```

동작 과정

1단계

`try` 안의 코드를 실행한다.

2단계

오류가 발생하면 즉시 실행을 멈춘다.

3단계

`except` 부분으로 이동한다.

5. 가장 기본적인 예외 처리 예제

```
try:
    num = int(input("숫자 입력: "))
    print(10 / num)

except:
    print("오류가 발생했습니다.")
```

실행 1

입력:

5

출력:

2.0

오류가 없으므로 `except` 실행 안 함.

실행 2

입력:

0

출력:

오류가 발생했습니다.

실행 3

입력:

abc

출력:

오류가 발생했습니다.

6. 특정 예외만 처리하기

모든 오류를 한 번에 처리하는 것보다
오류 종류마다 다르게 처리하는 것이 좋다.

형식

```
try:
    코드

except 예외이름:
    처리 코드
```

7. 여러 개의 예외 처리하기

```
try:
    num = int(input("숫자 입력: "))
    print(10 / num)

except ZeroDivisionError:
```

```
print("0으로 나눌 수 없습니다.")

except ValueError:
    print("숫자만 입력해야 합니다.")
```

8. 코드 흐름 자세히 이해하기

입력이 0인 경우

```
num = 0
```

```
print(10 / num)
```

에서 오류 발생.

따라서:

```
except ZeroDivisionError:
```

부분 실행.

입력이 abc인 경우

```
int("abc")
```

에서 오류 발생.

따라서:

```
except ValueError:
```

부분 실행.

9. Exception 사용하기

모든 예외의 부모 역할을 하는 것이 `Exception`

예시

```
try:
    num = int(input("숫자 입력: "))
    print(10 / num)

except Exception:
    print("예외 발생")
```

장점

모든 오류를 처리 가능

단점

어떤 오류인지 구분이 어려움

그래서 실제로는 구체적인 예외를 사용하는 경우가 많다.

10. 오류 메시지 출력하기

오류 내용을 직접 확인할 수도 있다.

형식

```
except 예외이름 as 변수:
```

예시

```
try:
    print(10 / 0)

except Exception as e:
    print("오류 내용:", e)
```

출력:

오류 내용: division by zero

11. else 문

예외가 발생하지 않았을 때만 실행된다.

형식

```
try:
    코드

except:
    예외 처리

else:
    정상 실행 시 코드
```

예시

```
try:
    num = int(input("숫자 입력: "))

except ValueError:
    print("숫자 입력!")

else:
    print("입력 성공")
```

흐름 이해

- 오류 발생 → except 실행
- 오류 없음 → else 실행

12. finally 문

예외 발생 여부와 관계없이 항상 실행된다.

형식

```
try:
    코드

except:
    코드

finally:
    항상 실행
```

예시

```
try:
    file = open("test.txt")

except FileNotFoundError:
    print("파일이 없습니다.")

finally:
    print("프로그램 종료")
```

finally는 언제 사용할까?

주로:

- 파일 닫기
- 프로그램 마무리 작업
- 데이터 저장

등에 사용한다.

13. 자주 사용하는 예외 종류

예외 이름	의미
ValueError	잘못된 값
TypeError	자료형 오류
ZeroDivisionError	0으로 나누기
IndexError	리스트 범위 초과
KeyError	딕셔너리에 없는 키
FileNotFoundError	파일 없음

14. raise - 일부러 예외 발생시키기

직접 오류를 만들 수도 있다.

예시

```
age = -5

if age < 0:
    raise ValueError("나이는 음수가 될 수 없습니다.")
```

왜 사용할까?

잘못된 입력을 강제로 막기 위해 사용.

15. 사용자 정의 예외

직접 새로운 오류 이름을 만들 수도 있다.

예시

```
class MyError(Exception):
    pass
```

사용

```
raise MyError("사용자 정의 오류")
```

예제 1 - 비밀번호 확인

```
password = "1234"

try:
    user = input("비밀번호 입력: ")

    if user != password:
        raise ValueError("비밀번호가 틀렸습니다.")

    print("로그인 성공")

except ValueError as e:
    print(e)
```

예제 2 - 리스트 안전하게 접근하기

```
numbers = [10, 20, 30]

try:
    index = int(input("인덱스 입력: "))
    print(numbers[index])

except IndexError:
    print("리스트 범위를 벗어났습니다.")

except ValueError:
    print("숫자를 입력하세요.")
```

16. 실무에서 예외 처리가 중요한 이유

실제 프로그램은 사용자가 어떤 입력을 할지 예측하기 어렵다.

예외 처리가 없으면:

- 앱 종료
- 게임 튕김
- 사이트 오류

등이 발생할 수 있다.

그래서 거의 모든 프로그램에서 예외 처리를 사용한다.

17. 많이 하는 실수

실수 1

```
except:
```

만 너무 많이 사용하기

→ 어떤 오류인지 알기 어려움

실수 2

오류를 무시하기

```
except:  
    pass
```

→ 디버깅 어려움

실수 3

try 범위를 너무 크게 잡기

```
try:  
    코드 엄청 많음
```

→ 어디서 오류 났는지 찾기 어려움

18. 핵심 정리

가장 중요한 문법

```
try:
    실행 코드

except 예외이름:
    오류 처리

else:
    오류 없을 때 실행

finally:
    항상 실행
```